Optical Binding Documentation

Overview

The optical binding simulation is divided into four main files:

- `constants.py`
- `functions.py`
- `main.py`
- `plotting_2D.py`
  Each file has a basic descriptor of what they do at the top of the file.

The most basic use case for the program is

1. Set the initial state you want (incident beam properties, position, particle properties, etc.) via `constants.py`
2. Determine which particles you want to pin in `main.py`
3. Run `main.py`. This will save `.npy` files in the `/data/` folder.
4. Run either `plotting_2D.py` or `scatter_plots.py` to generate visualizations.

Setting Initial Conditions

`constants.py` is the file where you set all the simulation parameters you want. Things like *polarization*, *2D vs 3D* ,particle properties (*permittivity*, *position*, etc.). The positions are governed by a variable called `init_pos_arr`:

```
# In constants.py
init_pos_arr = np.asarray(
    [
        [0, 1, 0],
        [1, 0, 0],
    ]
)
```

Where each element of the array is an x, y, z position. So above there are two particles placed in two separate locations. Note the default unit here is nanometers. *The default units in the simulation can be found at the top of the* `constants.py` *file under the units comment*.

The initial position can be set in several ways. Another way to set it is to use `np.random`.

```
# In constants.py
init_pos_arr = np.random.uniform(low=-L / 2, high=L / 2, size=(20, 3))
```

This generates 20 x, y, z positions with values between $(-L/2, L/2)$.

If you want to trap particles in the $z = 0$ plane (useful for 2D simulation) the following line will do that

```
# In constants.py
init_pos_arr[:, 2] = 0 # sets z = 0 for all particle positions
```

Pinning Particles (Seeding)

Once you have set the desired initial conditions and simulation parameters via `constants.py`. You can go into main and set how many particles you want to pin in place, if any. This is done by uncommenting the following code

```
# In main.py
 velocity_arr[step, 0:2] = np.asarray(
    [0, 0, 0]
 )  # set velocity to zero on the first two particles.
```

Note, 0:2 means 0 up to 2 but not including 2.

The reason that this code works is because it happens after the velocities are calculated, then overwrites their values with 0's.

Running the Program

Once you have set the parameters you want (usually only the initial position, polarization, and if any particles are pinned in place) you can run the simulation by running `main.py`.

`main.py` calls `functions.py` do perform most of the complicated physics. The basic structure of `main.py` is

1. Given the positions of the particles, calculate the incident field on them.
2. Calculate the self consistent dyadic Green's function and use it to find the induced dipole moments, p_i, for each particle.
3. Use p_i and the particle positions to calculate the forces acting on each particle.
4. Update the positions of the particle based on last steps velocity.
5. Calculate the current steps velocity (optionally setting some velocities to 0 to pin particles in place).
6. Repeat steps 2 - 5 until `maxsteps` is reached.

At the end of the program, a file named `positions.npy` will be saved as well as any other data explicitly saved.

Visualizing Results

Using the `positions.npy` file, saved in the `/data/` folder, `plotting_2D.py` or `scatter_plots.py` can be called.

`plotting_2D.py` will generate a series of `.png` files and then compile them into a video.

`scatter_plots.py` will generate a plethora of scatter plots based on different `.npy` files you can save with the `main.py`. This includes 3D plots.

How Data is Stored

All the data in this simulation is stored in numpy arrays. Before working with the code it is important to have a strong understanding of how numpy arrays work. For example, if you don't know what `A[:, None]` means a lot of the code will be incomprehensible. A good introduction to numpy arrays can be found here.

Much of the data is stored as a list of x, y, z positions for N particles. In `functions.py` all the functions attempt to have comments showing what shape to expect as an input and an output, as well as having line by line comments showing what the size of the object is at each step. The hardest part of this code is tracking the correct size of the objects.

The `None` keyword helps extend objects of a lower rank to enable operations with higher rank objects.